



Centro Universitario UAEM Zumpango

Licenciatura en Ingeniería en Computación

Sistemas operativos

Reporte técnico

Presenta:

Cristian Cruz Hernández

Introducción:

En este reporte documenta el proceso para cumplir con la tarea asignada en clase, no tome todas las capturas pero incluiré los pasos seguidos y las evidencias que tengo así como el programa creado en Python utilizando la librería de OpenCV. La idea principal de esta tarea es mostrar que cada comando tiene un por qué y su comprobación.

Herramientas utilizadas:

Para esta actividad utilice GitHub como plataforma de alojamiento para el repositorio, el CMD (consola de comandos), Git Bash para el alojamiento local del repositorio y Python para la creación del código utilizado.

Desarrollo:

1. Primero cree el repositorio desde GitHub con el nombre solicitado, quedando de la siguiente manera: **DevOps_CruzCristian**, se configuro público, sin archivo README ni otra cosa, dejando así un repositorio vacío para agregar el contenido posteriormente desde la terminal CMD.

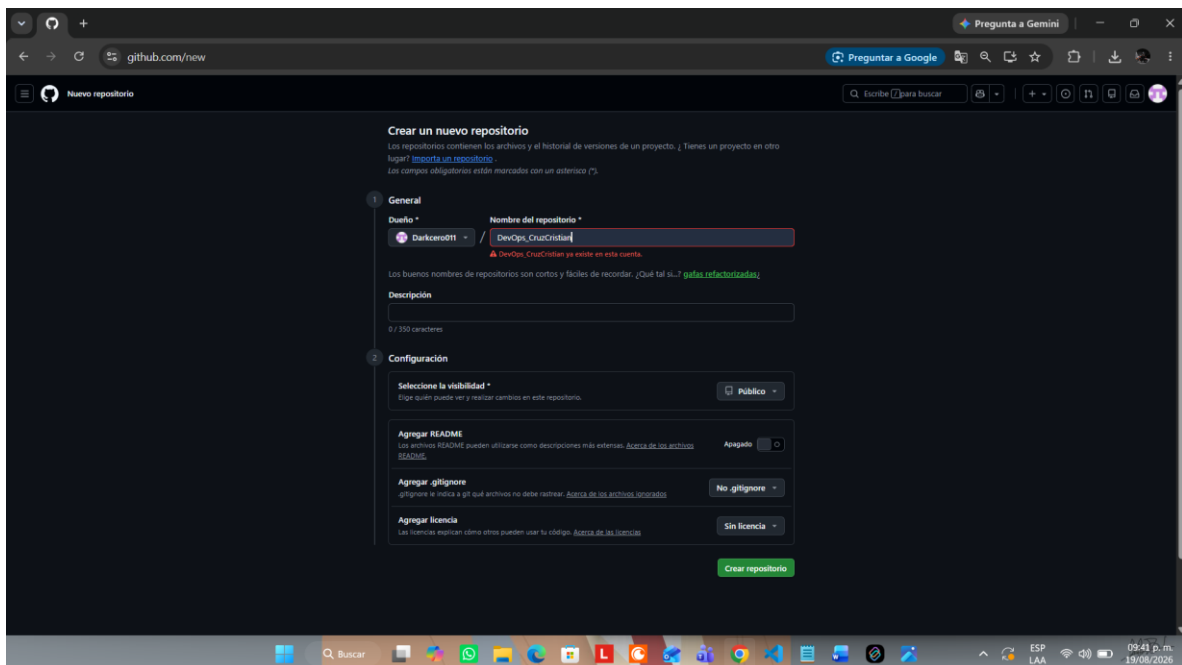
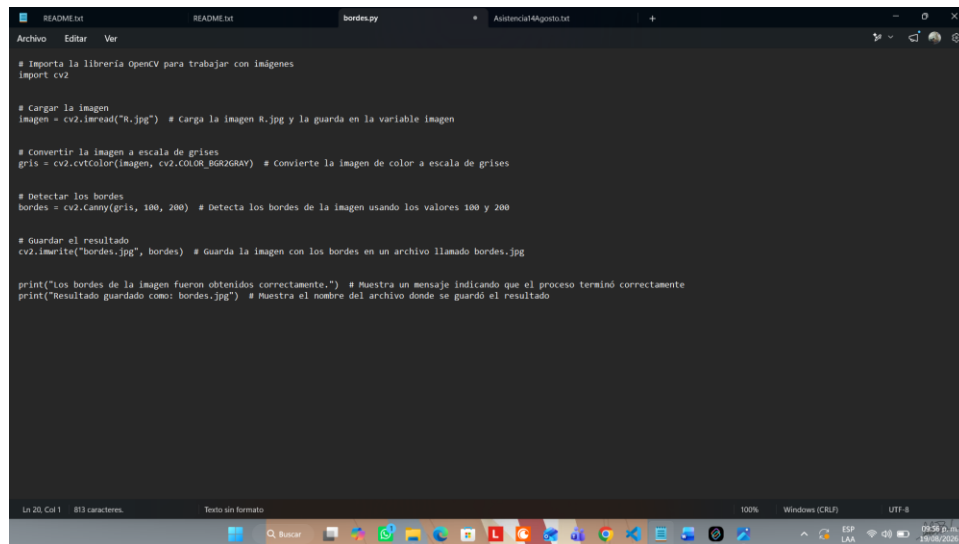


Imagen 1. Creación del repositorio en Github

2. Lo siguiente fue crear la carpeta de trabajo de manera local, desde mi CMD partí de la carpeta Documentos, utilice el comando **mkdir** para la creación de la carpeta con nombre **MiProyectoSO_CC**, después para confirmar la creación utilice el comando **dir**.

- Después de esto, dentro de la carpeta la inicialice como repositorio de Git usando **git init**, verifique usando **git status** y deje sin commits o archivos.
- Luego, para conectar el repositorio local con el creado en GitHub los vincule con **git remote add origin** seguido de la URL del repositorio **DevOps_CruzCristian** y comprobé la conexión mediante el comando **git remote -v**.
- Lo siguiente fue la creación del programa en Python para la detección de bordes de una imagen. Estando en la consola usé **Notepad** **bordes.py** para generar el archivo, estando ahí escribí el programa, este lo que hace es buscar y cargar la imagen con el nombre dado, luego la convierte a escala de grises y le aplica el algoritmo Canny para poder detectar sus bordes para después guardar la imagen con el nombre de “bordes.jpg”.



```

# Importa la libreria OpenCV para trabajar con imágenes
import cv2

# Cargar la imagen
imagen = cv2.imread("8.jpg") # Carga la imagen 8.jpg y la guarda en la variable imagen

# Convertir la imagen a escala de grises
gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY) # Convierte la imagen de color a escala de grises

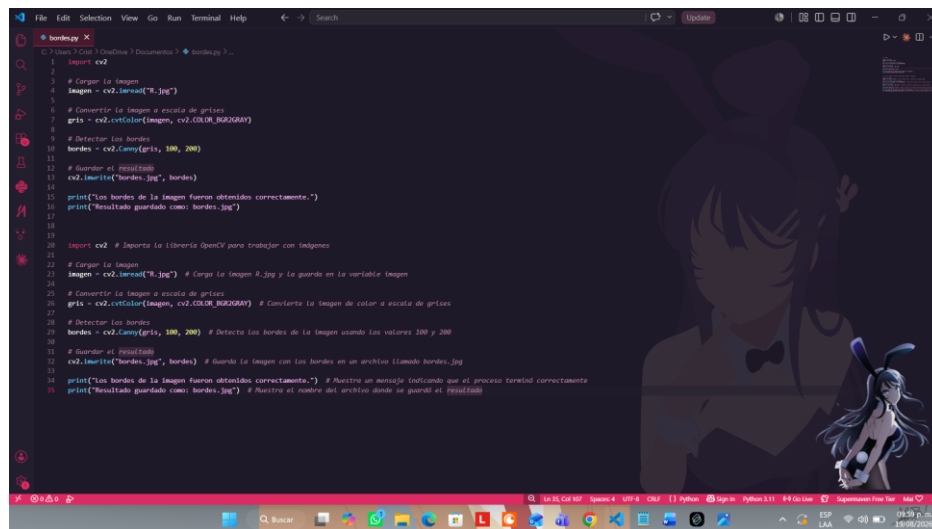
# Detectar los bordes
bordes = cv2.Canny(gris, 100, 200) # Detecta los bordes de la imagen usando los valores 100 y 200

# Guardar el resultado
cv2.imwrite("bordes.jpg", bordes) # Guarda la imagen con los bordes en un archivo llamado bordes.jpg

print("Los bordes de la imagen fueron obtenidos correctamente.") # Muestra un mensaje indicando que el proceso terminó correctamente
print("Resultado guardado como: bordes.jpg") # Muestra el nombre del archivo donde se guardó el resultado

```

Imagen 2. Código escrito del programa **bordes.py** en bloc de notas



```

1 import cv2
2
3 # Cargar la imagen
4 imagen = cv2.imread("8.jpg")
5
6 # Convertir la imagen a escala de grises
7 gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
8
9 # Detectar los bordes
10 bordes = cv2.Canny(gris, 100, 200)
11
12 # Guardar el resultado
13 cv2.imwrite("bordes.jpg", bordes)
14
15 print("Los bordes de la imagen fueron obtenidos correctamente.")
16 print("Resultado guardado como: bordes.jpg")
17
18
19
20 import cv2 # Importa la libreria OpenCV para trabajar con imágenes
21
22 # Cargar la imagen
23 imagen = cv2.imread("8.jpg") # Carga la imagen 8.jpg y la guarda en la variable imagen
24
25 # Convertir la imagen a escala de grises
26 gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY) # Convierte la imagen de color a escala de grises
27
28 # Detectar los bordes
29 bordes = cv2.Canny(gris, 100, 200) # Detecta los bordes de la imagen usando los valores 100 y 200
30
31 # Guardar el resultado
32 cv2.imwrite("bordes.jpg", bordes) # Guarda la imagen con los bordes en un archivo llamado bordes.jpg
33
34 print("Los bordes de la imagen fueron obtenidos correctamente.") # Muestra un mensaje indicando que el proceso terminó correctamente
35 print("Resultado guardado como: bordes.jpg") # Muestra el nombre del archivo donde se guardó el resultado

```

Imagen 3. Código en VScode

- Después, ejecute el script con Python `bordes.py` y la consola confirme la obtención de los bordes y guarde la imagen, se puede encontrar esa imagen guardada en la ubicación asignada, le cambie el nombre a **Result_CC** como se pedía en la tarea.

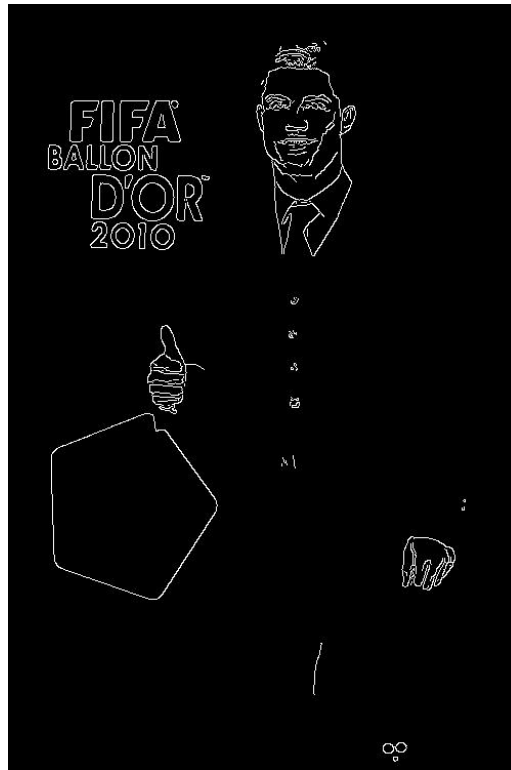


Imagen 4. Resultado de la detección de bordes

- Finalmente subí todo lo realizado al repositorio remoto, añadí los archivos con **git add**, los confirme usando **git commit -m "Programa para detectar los bordes de una imagen"**, use **git push** para enviarlos a GitHub y así los archivos quedaron reflejados en el repositorio **DevOps_CruzCristian** en GitHub.
Al final, el repositorio **DevOps_CruzCristian** contiene la carpeta de trabajo, el script **bordes.py**, la imagen original utilizada y la imagen resultante renombrada como **Result_CC.png**.

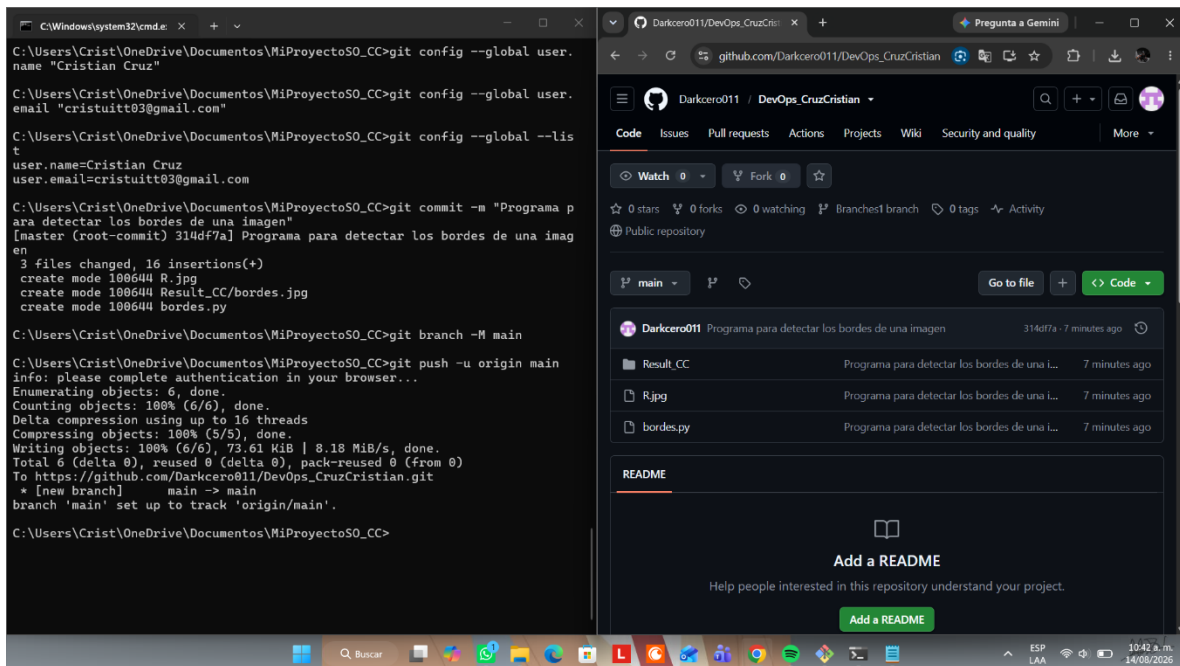


Imagen 5. Evidencia de la actividad con fecha y hora de su realización.

Conclusión:

La actividad realizada me ayudo a comprender el uso de comandos, el trabajo con GitHub y Git y la programación en Python dirigida a imágenes, me ayudo a ver la lógica de cada comando, su orden y su función, con esto pude absorber conocimientos nuevos que puedo aplicar en mi vida estudiantil y tal vez mas adelante en mi vida laboral.